

Prosit 3 – Complément du Prosit 2 avec MongoDB, Validation et Upload

Objectif

Compléter le Prosit 2 en :

- Ajoutant la persistance avec MongoDB
- Validant les données avec class-validator et class-transformer
- Utilisant des pipes pour transformer et valider les données
- Gérant l'upload de fichiers avec Multer

Instructions pour les étudiants

Installation des dépendances

1. Installez MongoDB et les dépendances :

```
npm install @nestjs/mongoose mongoose
```

2. Installez la validation et la transformation :

```
npm install class-validator class-transformer
```

3. Installez Multer pour la gestion des fichiers :

```
npm install @nestjs/platform-express multer
```

Configuration MongoDB

Connectez le projet à une base universite via MongooseModule.forRoot dans app.module.ts.

Modules

Vérifiez que vos modules utilisateurs et options importent maintenant MongooseModule.forFeature pour lier vos schémas aux modèles.

Schémas Mongoose

Créez un schéma Utilisateur avec :

- studentId : string, obligatoire
- age : number, obligatoire

- firstName : string, obligatoire
- lastName : string, obligatoire
- email : string, obligatoire et unique

- avatar : string
- Activez timestamps

Créez un schéma Option avec :

- name : string, obligatoire

DTO et Validation

Pour chaque DTO (CreateUtilisateurDto, UpdateUtilisateurDto, CreateOptionDto, UpdateOptionDto) :

- Utilisez les decorators @IsString(), @IsEmail(), @IsOptional(), etc. de class-validator.
- Transformez les entrées en types corrects avec class-transformer (@Type() si nécessaire).

Pipes

Utilisez le ValidationPipe global dans main.ts :

```
app.useGlobalPipes(new ValidationPipe({ whitelist: true, transform: true }));
```

Cela permet de valider et transformer automatiquement toutes les entrées des requêtes.

Services CRUD

Modifiez vos services pour utiliser Mongoose avec :

- create()
- findAll()
- findOne(id)
- update(id, dto)
- remove(id)

Gérer les cas où l'ID n'existe pas avec NotFoundException.

Contrôleurs REST

Conservez les endpoints existants (POST, GET, PATCH, DELETE) pour utilisateurs et options.

Ajoutez un endpoint pour upload de fichier (exemple : /utilisateurs/:id/avatar) avec Multer :

```
@Post('/:id/avatar')
@UseInterceptors(FileInterceptor('file'))
uploadFile(@Param('id') id: string, @UploadedFile() file: Express.Multer.File) {
  return { message: 'Fichier uploadé', filename: file.originalname };
}
```

Gestion des erreurs

Implémentez un `HttpExceptionHandler` global qui renvoie toutes les erreurs au format JSON :

```
{  
  "statusCode": 404,  
  "message": "Utilisateur non trouvé"  
}
```

Résultat attendu

Une API REST fonctionnelle reliée à MongoDB, avec :

- Validation automatique des DTO
- Transformation des données avec pipes
- Upload de fichiers
- Middleware Logger
- Gestion d'erreurs propre et standardisée